Year	Research Paper	Methodology	Research Gap / Limitation	How Your Project Improves It
1998	**Duckworth & Lewis – A Fair Method for Resetting the Target in Interrupted One-Day Cricket Matches**	Statistical resource model based mainly on **overs remaining and wickets lost** to estimate remaining scoring resources. (Taylor & Francis Online)	Uses an average/generalized model. It does not use detailed ball-by-ball player, team, venue or matchup information.	Your project uses **ball-by-ball data**, current score, wickets, run rate, required run rate, balls remaining, player form and match context to create a more detailed match state.
2004	**Dean & Ghemawat – MapReduce: Simplified Data Processing on Large Clusters**	Introduced the **MapReduce distributed processing model**, where Map and Reduce operations process huge datasets across multiple machines. (USENIX)	MapReduce is a general-purpose processing framework and does not provide cricket-specific analytics or machine-learning functionality.	Your project applies distributed processing specifically to **10M+ cricket deliveries**, using MapReduce for bowling analytics and Spark for advanced processing.
2009	**Thusoo et al. – Hive: A Warehousing Solution Over a Map-Reduce Framework**	Developed **Hive**, providing SQL-like querying over large datasets stored in Hadoop.	Hive is primarily a batch/query layer and is not designed as the main engine for iterative ML or real-time match-state prediction.	Your project uses **Hive for analytical queries**, while PySpark/Spark MLlib handles feature engineering and machine learning.

2011–2012	**Brooker & Hogan – WASP (Winning and Score Predictor)**	Uses historical transition probabilities and **dynamic programming** to estimate expected scores and win probability at different match states.	WASP uses generalized historical conditions and does not provide the full distributed Big Data architecture used in your project.	Your project calculates **ball-level match states** using a large multi-format dataset and combines them with **HDFS + MapReduce + Hive + Spark MLlib**.
2012	**Zaharia et al. – Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing**	Introduced **Spark/RDDs**, enabling distributed, fault-tolerant in-memory processing and faster iterative workloads.	Spark itself is a general-purpose processing framework; it does not provide cricket-specific features or prediction models.	Your project uses **PySpark specifically for cricket ETL, rolling player form, match-state generation and large-scale feature engineering**.
2018	**Passi & Pandey – Increased Prediction Accuracy in the Game of Cricket Using Machine Learning**	Compared **Naive Bayes, Random Forest, SVM and Decision Tree** for predicting player performance such as runs and wickets.	Primarily uses static/pre-match aggregate statistics and does not update predictions as the match progresses.	Your project uses **ball-by-ball match-state data**, allowing predictions to change after every delivery rather than only before the match.

2020	**Chakwate & Madhan – Analysing LSTM Models for Cricket Match Outcome Prediction**	Uses an **LSTM/RNN** with ball-wise features to continuously estimate win probability during a match. (arXiv)	Ball-by-ball prediction is addressed, but the work is not presented as a distributed HDFS/MapReduce/Hive pipeline and is evaluated in a more limited setting.	Your project combines **ball-level prediction with distributed Big Data infrastructure** and processes a much broader multi-competition dataset.
2021	**Awan et al. – Cricket Match Analytics Using the Big Data Approach**	Uses **Spark ML** and conventional ML to predict ODI team scores using cricket data from Cricsheet.	Focuses primarily on **score prediction** and does not combine phase-aware analytics, ball-by-ball win probability, HDFS, Hive and MapReduce into one pipeline.	Your project extends this toward **win-probability prediction + player analytics + phase analysis + HDFS + MapReduce + Hive + Spark MLlib**.
2022	**Wickramasinghe – Applications of Machine Learning in Cricket: A Systematic Review**	Systematic review of **72 cricket ML studies** from 2001–2021. It identifies feature engineering as an important factor in prediction performance.	The literature is heavily focused on pre-match/aggregate prediction; ball-by-ball and infrastructure-scale work are comparatively limited.	Your project focuses directly on **ball-by-ball feature engineering at large scale**, using Spark to construct match-state and player-performan ce features.

2024	**Chakraborty et al. – Cricket Data Analytics: Forecasting T20 Match Winners Through Machine Learning**	Compares **Logistic Regression, SVM, Random Forest, Decision Tree, XGBoost and Voting Classifier** for T20 winner prediction.	Uses pre-match features such as ranking, form and head-to-head records; the study does not fully model the match progression ball by ball.	Your project performs **in-match prediction** using score, wickets, run rate, required run rate, balls remaining and phase information.

1. 1998 — Duckworth & Lewis

Paper

A Fair Method for Resetting the Target in Interrupted One-Day Cricket Matches

Methodology

The Duckworth-Lewis method models a cricket innings using two important resources:

- Overs remaining
- Wickets remaining

Historical scoring data is used to estimate how much scoring potential remains for a team.

Research Gap

The model essentially represents the match using a relatively small generalized state.

It does not consider:

- Individual batter strength
- Individual bowler strength
- Batter-bowler matchup
- Venue
- Recent player form
- Detailed ball-by-ball patterns

our Improvement

Your project takes the concept of **"match state"** further.

Instead of only:

> Overs remaining + wickets remaining

you use:

> **Current score + wickets lost + overs completed + run rate + required run rate + balls remaining + phase + player/matchup features**

This gives you a much richer representation of the current match state.